

A. Artifact Appendix

A.1 Abstract

This artifact contains the source code, build scripts, and evaluation scripts needed to reproduce two crucial results for FlexiCache. To run the experiments, an x86_64 Linux machine with an NVIDIA H100 GPU (94GB recommended, 80GB minimum), at least 256GB of host memory, and PCIe Gen5 CPU–GPU interconnect is required. A working CUDA development environment is also required to build FlexiCache, which is implemented on top of vLLM, from source. Reviewers can validate the artifact by cloning the GitHub repository, building the system, and running two shell scripts that generate CSV files of the results.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Stability-aware hierarchical KV-cache management and sparse decode attention.
- **Program:** LongBench evaluation scripts and throughput benchmarking scripts for FlexiCache and vLLM.
- **Compilation:** Python package build via `pip install -e .`, compiling custom CUDA, Triton, and C++ extensions for FlexiCache on top of vLLM.
- **Model:** meta-llama/Llama-3.1-8B-Instruct, downloaded automatically from Hugging Face when running the scripts; requires Internet access and a configured Hugging Face account.
- **Data set:** zai-org/LongBench, downloaded automatically from Hugging Face when running the scripts; requires Internet access.
- **Run-time environment:** Linux, Conda, Python 3.12, PyTorch 2.6.0+cu124, Triton 3.2.0, Transformers 4.50.0, CUDA compatible NVIDIA driver.
- **Hardware:** An x86_64 Linux machine with an NVIDIA H100 GPU (94GB recommended, 80GB minimum), at least 256GB of host memory and PCIe Gen5 CPU–GPU interconnect.
- **Run-time state:** For reliable throughput measurements, experiments should be run on an idle machine without other workloads contending for the GPU, CPU, RAM, or host–device bandwidth.
- **Execution:** Single-GPU execution via automated bash scripts. The artifact uses a large pinned host-memory pool (about 180GB by default) and frequent host–device KV-cache transfers during execution.
- **Metrics:** LLM generation throughput (tokens/second). Long-context benchmarks accuracy.
- **Output:** Accuracy numbers and generation throughput presented in 2 CSV files.
- **Experiments:** (1) LongBench accuracy retention compared to dense attention baseline (2) end-to-end throughput gain compared to vLLM baseline.
- **How much disk space required (approximately)?:** 100GB.
- **How much time is needed to prepare workflow (approximately)?:** Around 1 hour to build FlexiCache (implemented on top of vLLM) from source

- **How much time is needed to complete experiments (approximately)?:** Around 2 hours for the accuracy retention experiment. Around 3 hours for the throughput experiment.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** Apache License 2.0.
- **Data licenses (if publicly available)?:** MIT
- **Workflow framework used?:** Bash scripts and Python evaluation code.
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.18910205>

A.3 Description

A.3.1 How delivered

The artifact is delivered through the public GitHub repository:

<https://github.com/NazmulTakbir/FlexiCache>

The repository contains FlexiCache implementation and the evaluation scripts. Reviewers are expected to clone the repository, build the project, and run two provided shell scripts.

A.3.2 Hardware dependencies

The artifact is intended to run on a single x86_64 Linux machine with:

- 1× NVIDIA H100 GPU (94GB recommended, 80GB minimum),
- at least 256GB of host DRAM, able to allocate 180GB of pinned host-memory for KV cache offloading

A.3.3 Software dependencies

- Linux (we tested on Ubuntu 24.04.2 LTS),
- Anaconda (we used 24.9.2),
- Python 3.12,
- PyTorch 2.6.0+cu124,
- Triton 3.2.0,
- Transformers 4.50.0,
- Datasets 3.6.0,
- CUDA NVIDIA driver (we tested on CUDA 12.8).

A.3.4 Data sets

zai-org/LongBench. Publicly available. Downloaded automatically from Hugging Face when running the scripts; requires Internet access.

A.4 Installation

We recommend using Anaconda to isolate the software dependencies. Installation takes approximately 1 hour, primarily due to building vLLM from source.

```
# 1. Clone the repository
```

```

git clone https://github.com/NazmulTakbir/
  FlexiCache.git
cd FlexiCache

# 2. Create and activate a Conda environment
conda create -n FlexiCache python=3.12 -y
conda activate FlexiCache

# 3. set CUDA environment variables so nvcc is
  found during build (paths may vary by machine)
export CUDA_HOME=/usr/local/cuda-12.8
export CUDACXX=/usr/local/cuda-12.8/bin/nvcc
export PATH=/usr/local/cuda-12.8/bin:$PATH

# 4. Build from source
export MAX_JOBS=10
pip install -e . # 30min to 1 hour

# 5. Install additional dependencies
pip install -r flexicache_requirements.txt

# 6. Authenticate with Hugging Face (Required for
  Llama 3.1). Must also accept the model
  agreement on Hugging Face
pip install huggingface_hub
huggingface-cli login

```

A.5 Experiment workflow

We provide two push-button bash scripts to reproduce the primary accuracy and throughput results for meta-llama/Llama-3.1-8B-Instruct. Both commands should be run from the repository root after installation.

A.5.1 Accuracy on LongBench

This script evaluates FlexiCache on a subset of 8 LongBench tasks.

```

# Run
bash FlexiCache/LongBench.sh Llama8b

# Results are written to:
benchmarks/FlexiCache/Language_Modelling/LongBench
  /longbench_table.csv

```

This workflow takes approximately 2 hours.

A.5.2 Throughput

This script benchmarks end-to-end throughput (tokens/sec) of FlexiCache (with a 1024-token budget) against baseline vLLM for output lengths of 100, 500, 1000, and 1500 tokens, using input prompts randomly sampled between 10k and 30k tokens.

```

# Run
bash FlexiCache/Throughput.sh Llama8b

# Results are written to:
benchmarks/FlexiCache/Throughput/throughput_table.
  csv

```

This workflow also takes approximately 3 hours.

A.6 Evaluation and expected result

In this section, we provide reference results for comparison. Due to differences in machine environments, absolute throughput values may vary. However, the relative speedup of FlexiCache over the vLLM baseline should remain consistent. Accuracy results may also show small deviations from the reported numbers due to the inherent stochasticity of LLM generation and minor floating-point differences across hardware.

A.6.1 Accuracy on LongBench

Task	Dense	FlexiCache
Qasper	45.23	45.92
MultiField-en	54.93	54.94
2WikiMQA	45.45	45.32
Musique	30.18	31.22
GovReport	34.98	34.22
MultiNews	27.18	27.10
TriviaQA	91.64	91.49
RB-P	49.43	49.70
Avg. Ratio	–	1.00

Table 1. Accuracy Retention on LongBench

A.6.2 Throughput (tokens/second)

Output Length	vLLM	FlexiCache-1024	Speedup
100	11178.7	12433.1	1.11×
500	7742.1	11480.2	1.48×
1000	5609.9	9965.4	1.78×
1500	4549.0	9444.9	2.08×

Table 2. Throughput Gain

A.7 Experiment customization

```

# Run LongBench & Throughput for other models
bash FlexiCache/LongBench.sh Mistral7b
bash FlexiCache/LongBench.sh Mistral24b
bash FlexiCache/Throughput.sh Mistral7b
bash FlexiCache/Throughput.sh Mistral24b

```

A.8 Methodology

Submission, reviewing and badging methodology:

- <http://cTuning.org/ae/submission-20190109.html>
- <http://cTuning.org/ae/reviewing-20190109.html>
- <https://www.acm.org/publications/policies/artifact-review-badging>